The Journal of Open Source Software

# TMMax: High-performance modeling of multilayer thin-film structures using transfer matrix method with JAX

**Bahrem Serhat Danis** [1] **and Esra Zayim** [2]

**1** Department of Electrical and Electronics Engineering, Koç University, Sariyer, Istanbul, 34450, Turkey
**2** Physics Engineering Department, Istanbul Technical University, Istanbul, 34469, Turkey

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:**

**Submitted:** 15 July 2025
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

## Abstract

Optical multilayer thin-films are fundamental components that enable the precise control of reflectance, transmittance, and phase shift in the design of photonic systems. Rapid and accessible simulation of these structures holds critical importance for designing and analyzing complex coatings, such as distributed Bragg reflectors, anti-reflection layers, and spectral filters. While researchers commonly use the traditional transfer matrix method for designing these structures, its scalar approach to wavelength and angle of incidence causes redundant recalculations and inefficiencies in large-scale simulations. Furthermore, traditional method implementations do not support automatic differentiation, which limits their applicability in gradient-based inverse design approaches. Here, we present TMMax, a Python library that fully vectorizes and accelerates transfer matrix method using the high-performance machine learning library JAX. TMMax supports CPU, GPU, and TPU hardware, includes a publicly available material database, and offers comprehensive multilayer optical thin-film analysis tools. Our approach, demonstrated through benchmarking, allows us to model thin-film stacks with hundreds of layers within seconds. This illustrates that our method achieves a simulation speedup of x100s over a baseline NumPy implementation, providing a significant advantage in computational efficiency. Our method enables rapid and scalable simulations of large and complex multilayer thin-film structures, significantly accelerating the design process.

## Statemement of need

Transfer Matrix Method (TMM) is a technique used to model multilayer optical thin-films, where the path of light incident on the film is determined using Snell's law, and the transmittance and reflectance coefficients at interfaces between different materials are calculated using the Fresnel equations.

$$\mathbf{M} = \mathbf{I}_0 \cdot \prod_{i=1}^{N-2} \mathbf{M}_i \tag{1}$$

$$\mathbf{M}_i = \mathbf{I}_i \mathbf{P}_i = \begin{bmatrix} \alpha_{i,i+1} & \gamma_{i,i+1} \\ \gamma_{i,i+1} & \alpha_{i,i+1} \end{bmatrix} \begin{bmatrix} e^{-j\delta_i} & 0 \\ 0 & e^{j\delta_i} \end{bmatrix} = \begin{bmatrix} \alpha_{i,i+1} e^{-j\delta_i} & \gamma_{i,i+1} e^{j\delta_i} \\ \gamma_{i,i+1} e^{-j\delta_i} & \alpha_{i,i+1} e^{j\delta_i} \end{bmatrix} \tag{2}$$

In TMM, the optical behavior of a multilayer structure composed of dielectric materials is obtained by computing the system matrix $\mathbf{M}$, as shown in Equation (1). This matrix calculation, commonly referred to as the Abeles TMM (Abelès, Florin, 1950), results from the successive multiplication of the transfer matrices of each layer and interface (Katsidis & Siapkas, 2002)

as expressed in Equation (2). Each layer is represented as $\mathbf{M}_i = \mathbf{I}_i \mathbf{P}_i$, where $\mathbf{I}_i$ denotes the interface matrix and $\mathbf{P}_i$ represents the propagation matrix that describes the light propagation through the $i$th layer. The interface matrix $\mathbf{I}_i$ contains terms that depend on the reflection and transmission coefficients at the boundary between the $i$th and $(i+1)$th layers.

$$\alpha_{i,i+1} = \begin{cases} \dfrac{n_i \cos\theta_i + n_{i+1}\cos\theta_{i+1}}{2n_i \cos\theta_i} & \text{(s-pol.)} \\ \dfrac{n_i \cos\theta_{i+1} + n_{i+1}\cos\theta_i}{2n_i \cos\theta_i} & \text{(p-pol.)} \end{cases} \tag{3}$$

$$\gamma_{i,i+1} = \begin{cases} \dfrac{n_i \cos\theta_i - n_{i+1}\cos\theta_{i+1}}{2n_i \cos\theta_i} & \text{(s-pol.)} \\ \dfrac{n_i \cos\theta_{i+1} - n_{i+1}\cos\theta_i}{2n_i \cos\theta_i} & \text{(p-pol.)} \end{cases} \tag{4}$$

$$\delta_i = \frac{2\pi}{\lambda} n_i d_i \cos\theta_i \tag{5}$$

As can be seen in Equation (3) and Equation (4), its elements, $\alpha_{i,i+1}$ and $\gamma_{i,i+1}$, vary depending on the polarization of the incoming light. The propagation matrix $\mathbf{P}_i$ characterizes the optical phase accumulated by the electromagnetic wave as it traverses the $i$th layer. As detailed in Equation (5), the accumulated phase $\delta_i$ is given by $\delta_i = \frac{2\pi}{\lambda} n_i d_i \cos\theta_i$, where complex-valued $n_i$ is the sum of the refractive index and extinction coefficient of the layer, $d_i$ is the layer thickness, $\theta_i$ is the angle of incidence, and $\lambda$ is the wavelength of the incoming light (Macleod & Macleod, 2010). Finally, the elements of the resulting system matrix $\mathbf{M}$ yield the reflection and transmission properties of the optical coating formed by the stack of layers (Harbecke, 1986). The TMM relies fundamentally on matrix–matrix multiplications and linear transformations, both of which are computationally efficient operations (Dyakov et al., 2010). Owing to the inherently structured nature of these calculations (Santbergen et al., 2013), the overall simulation workflow benefits substantially from hardware-level acceleration via vectorization and parallelization. This makes TMM particularly well suited for high-throughput modeling (Centurioni, 2005) and optimization of multilayer optical coatings, where rapid evaluation across a wide parameter space is essential (Shi et al., 2017). However, in currently implemented TMM packages (Byrnes, 2020; Dmitriev, 2017; Dominec, 2017), the angle of incidence and wavelength of the incoming light are typically treated as scalar inputs, rather than as vectorized parameters.

The simulation of the stack of layers, as shown in Figure 1a, is performed by taking only a single wavelength and angle of incidence value, as illustrated in Figure 1b, and consequently, in an traditional TMM implementation, a single function call returns transmittance, reflectance, and absorbance for only one wavelength and one angle of incidence. In scalar implementations of the TMM, two nested loops are typically employed to simulate the optical response of thin-films over arrays of wavelengths and angles of incidence (Byrnes, 2020). In addition to this, redundant computations frequently arise. For example, when investigating the optical characteristics of a thin-film at multiple angles of incidence for a fixed wavelength, the wavelength-dependent component of the wavevector along the propagation direction, $k_z$, is recalculated. This approach is highly inefficient, as these redundant recalculations significantly increase computational cost without improving accuracy. To eliminate these redundant calculations, wavelength and angle of incidence should be considered as vectors rather than scalars, and by performing vectorization, the two nested for loops can be eliminated. In TMMax, we vectorize wavelength and angle of incidence using the JAX library (Bradbury et al., 2018). As seen in the schematic of the vectorized implementation in Figure 1c, we vectorize all intermediate operations in TMM and subsequently apply the JAX's just-in-time (JIT) decorator. Instead of running the mapped TMM code sequentially over each batch element of wavelength and angle of incidence, jax.jit fuses all operations across the batch into a single XLA-compiled (OpenXLA Team, 2023) kernel. This reduces function call overhead and provides a faster TMM implementation.

The Journal of Open Source Software

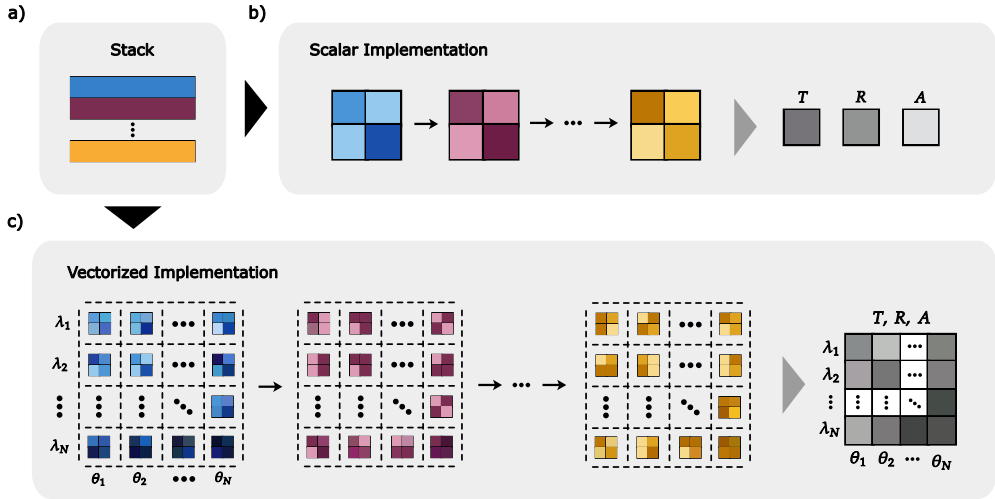


**Figure 1:** Schematic representation of two implementation strategies for calculating transmission, reflection, and absorption in multilayer thin-film simulations. A multilayer system composed of stacked layers (a) is modeled by traditionally computing the optical response sequentially through multiplication of a chain of 2×2 transfer matrices for each wavelength and angle of incidence (b). In contrast, the vectorized approach (c) performs the same computation by vectorizing the operations along both the wavelength and angle-of-incidence axes.

The chain of matrices is multiplied using a for loop to obtain the overall system matrix in the TMM, as expressed in Equation (1). However, the for loop inherently involves carry-over dependencies that limit computational efficiency in calculating the system matrix; fortunately, more advanced computational strategies can be used to mitigate such inefficiencies (Nishida et al., 2011). In TMMax, we eliminate this carry-over for loop by employing the scan function within the lax module of the JAX. The lax module is a Python wrapper that enables the execution of primitives written in XLA within Python. In this way, we achieve a faster TMM by removing the for loop from the code, vectorizing the computations, applying JIT compilation transformations, and thus preventing the TMM from entering the for loop, thereby avoiding interpreter bottlenecks. Moreover, most of the implemented TMM libraries operate on CPUs and the computation of the system matrix over a specified wavelength and angle of incidence range typically relies on sequential for loops, which iterate across the spectral and angular domains to evaluate transmittance and reflectance. This inherently limits computational efficiency due to the serial nature of the execution. While this sequential execution remains efficient for thin-films with a limited number of layers, its performance degrades considerably as the layer count increases. For example, simulating a 100-layer structure becomes computationally intensive due to backend bottlenecks, where the prolonged runtime of for loops leads to significant processing delays. However, these delays can be effectively mitigated in TMM because calculations for different angles of incidence and wavelengths are inherently parallelizable and non-recursive. Therefore, TMM is inherently suitable for parallelization and can run on platforms such as GPUs and TPUs. For this reason, we implement TMMax with the JAX, thereby obtaining a backend-agnostic TMM. In other words, we enable TMMax to run seamlessly on CPUs, GPUs and TPUs without any code modification. Additionally, we facilitate deep learning–based inverse design involving TMM by performing all calculations on the GPU, eliminating the need for data transfer back to the CPU (Hegde, 2019).

Automatic differentiation holds a crucial role in optimization problems because it computes derivatives required for optimization both accurately and efficiently, unlike finite difference methods which approximate derivatives and can suffer from truncation and round-off errors (Baydin et al., 2018). This approximation arises because finite differences estimate derivatives by evaluating small perturbations in parameters, which inherently limits precision. In the design

process of systems such as multilayer thin-films, minimizing the loss function over physical parameters (thickness, refractive index, etc.) is necessary (Luce et al., 2022), and gradients play a critical role at this stage. Conventional TMM implementations are written using libraries like NumPy (Harris et al., 2020) that do not natively support automatic differentiation. For example, if automatic differentiation is desired in such NumPy-based implementations, an additional library such as Autograd (Maclaurin et al., 2015) must be used, which increases the implementation complexity of the code. TMMax, however, is designed according to the functional programming principles with JAX, enabling effortless gradient computation. Furthermore, TMMax can be integrated with the Optax library (DeepMind et al., 2020), a gradient processing and optimization library built on JAX that offers a wide range of optimization functions and procedures. This integration enables direct computation of derivatives with respect to thin-film parameters, such as layer thicknesses and material properties, thereby facilitating efficient gradient-based inverse design. As a result, these gradients can be utilized by advanced optimization algorithms, such as L-BFGS (Liu & Nocedal, 1989) and Adam (Kingma & Ba, 2017), to rapidly converge toward optimal thin-film configurations. Consequently, layered structures can be rapidly designed to achieve targets such as optical reflectivity or transmittance.

The majority of implemented TMM packages focus exclusively on the calculation of the system matrix (Acquaroli, 2022; Luce et al., 2022). However, these libraries generally lack a material database or offer only limited material data. This deficiency requires users to manually add their own refractive index and extinction coefficient data, which is both time-consuming and prone to errors. Furthermore, the accuracy of calculations performed with TMM directly depends on the reliability of the refractive index and extinction coefficient data used; if the material data is inaccurate, the simulation deviates from the experimentally obtained optical response. Therefore, in TMMax, we integrate a database of 30 widely used materials for multilayer optical thin-film fabrication. These materials are selected based on commonly used ones referenced in the reference book Thin-Film Optical Filters by H. Angus Macleod (Macleod & Macleod, 2010). The refractive index data were carefully gathered from refractiveindex.info (Polyanskiy, 2024), a trusted online database that collects optical properties from many scientific papers and verifies each entry against the original sources to ensure accuracy and reliability.The source of the material information for each study is documented in the "README.md" file located in the "docs/database_info" folder of the library, and we designed the TMMax material database as a dynamic, living ecosystem. To facilitate collaborative growth, we implemented user-friendly helper functions that support the straightforward addition of refractive index and extinction coefficient data from external laboratories into the TMMax database. By contributing data through these tools and submitting pull requests to the TMMax GitHub repository, we foster a community-driven platform that continually expands and enriches shared material knowledge for the thin-film coating community.

# Code structure

We designed TMMax to provide a comprehensive library that enables rapid simulations of multilayer optical thin-films, while also integrating a publicly accessible material database and implementing extensive functionalities to facilitate the evaluation and characterization of thin-film structures. As shown in Figure 2, we structured TMMax into three main components: the nk_data module, which serves as a curated material database; the optical properties computation core, responsible for all intermediate calculations required by the TMM; and the thin-film analysis interface, which enables performance evaluation and characterization of multilayer structures. Since TMMax is written in JAX, these components adhere to functional programming principles. The design does not use classes or object-oriented structures; however, a user who prefers object-oriented design can easily encapsulate TMMax functions within their own classes.

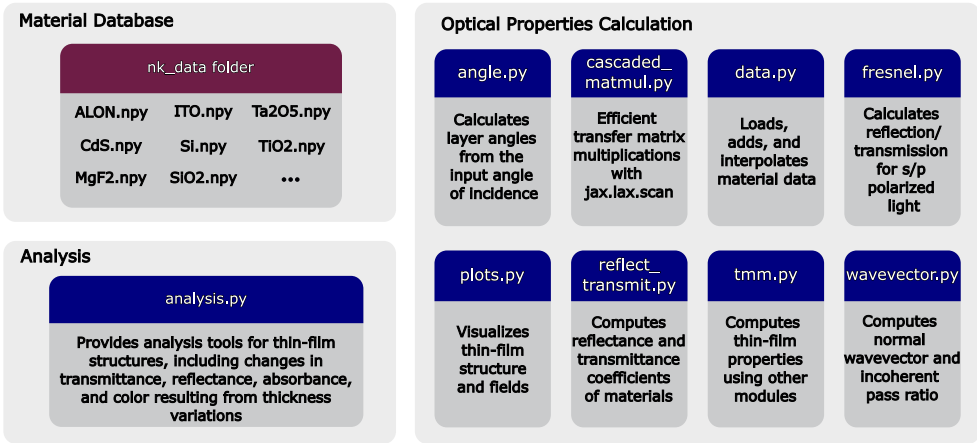


**Figure 2:** Modular architecture of TMMax, structured into three main components to facilitate efficient multilayer thin-film simulations. The Material Database (top left) comprises a curated set of .npy files containing refractive index and extinction coefficient data stored in the nk data folder. The Optical Properties Calculation core (right) includes modular scripts for angle calculations, transfer matrix multiplications, Fresnel reflection or transmission, and material interpolation, among others, together performing all intermediate computations essential for the TMM. The Analysis section (bottom left) provides tools to evaluate the optical performance of thin-film stacks, including transmittance, reflectance, absorbance, and resulting color as functions of layer thickness.

## Material Database

Dielectric materials used in thin-film designs are selected from a curated material database that contains refractive index and extinction coefficient data. These two optical data are obtained from the refractiveindex.info database of optical constants, which compiles data from numerous different scientific publications, carefully gathered from validated experimental studies and peer-reviewed publications to ensure optical accuracy across a wide spectral range. In addition, a reference file is available in the docs/database_info folder within TMMax GitHub repository, containing information for each material about the source publication and the measurement methods employed. Each material entry in the database is stored as a matrix in the .npy format with dimensions (3, N), where the first row contains wavelength values, and the second and third rows correspond to the refractive index and extinction coefficient values at those wavelengths, respectively. This storage format allows us to directly load data using JAX without any intermediate conversion, significantly facilitating the data loading process. Moreover, users can easily add new materials to the TMMax database. To support this, a Jupyter notebook in the docs/examples folder provides step-by-step instructions on how to add materials using TMMax's save_nk_data function. Users who wish to share their own materials can contribute by submitting a pull request to the TMMax GitHub repository.

## Optical Properties Calculation

Optical properties of the thin-film, such as transmittance, reflectance, and absorbance, are calculated in this section. As shown in Figure 2, this section contains eight different Python files that enable the implementation of the TMM specifically for coherent and incoherent thin-films. In TMMax, users simulate multilayer optical thin-films exclusively through the core TMM function implemented in the tmm.py module, while all other modules are specifically designed to handle intermediate computational processes required for an accurate and efficient execution of the TMM. To explain them sequentially, we calculate the angle between the light ray and the layer normal in each layer of a thin-film using the functions in angle.py. We use these angles together with the Fresnel equations implemented in fresnel.py within

DRAFT

the reflect_transmit.py file to calculate the reflection and transmittance coefficients between layers, considering both s- and p-polarizations. While performing polarization-dependent calculations, conditional statements such as if, which are typically used when running code through the Python interpreter, are avoided. Instead, the jax.lax.cond function is employed to ensure compatibility with JIT compilation. This prevents interpreter-related bottlenecks after transformation and enables efficient, polarization-specific coefficient evaluation within the TMM function. Additionally, functions necessary to compute the normal wavevector, which is required to determine the optical phase accumulation within each layer, and functions to calculate the transmittance and reflectance ratios of thick incoherent layers are located in the wavevector.py file. Ultimately, to combine these and find the system matrix, multiplication of chain matrices is required, and this chain matrix multiplication code is implemented in cascaded_matmul.py using the jax.lax.scan function. In addition to the intermediate steps of TMM, helper functions for data preprocessing and visualization are also essential. The first of these involves loading and interpolating refractive index and extinction coefficient data from the material database, which is implemented in the data.py file. The second involves visualizing the layer-by-layer material structure of the thin-film and observing the distribution of the electric field intensity between layers; these are implemented in the plot.py file.

### Analysis

The prediction of reflected color from multilayer optical thin-films under varying illumination conditions, along with the calculation of transmittance and reflectance sensitivities, is implemented within a single-file module. To observe the color of a multilayer thin-film, users can employ the functions within the analysis file, which leverage the Python-based ColorPy library (Kness, 2008) to perform accurate physical color calculations. Additionally, during fabrication, unavoidable variations occur from the target thicknesses in one or more layers of the multilayer structure, leading to changes in transmittance, reflectance, and consequently the resulting color of the thin-film. The analysis file implements functions that calculate how percentage deviations from these target thicknesses influence the film's optical properties, enabling sensitivity analysis for transmittance, reflectance, and color.

## Benchmarks

In multilayer optical thin-film simulations using the TMM, evaluating the runtime under different configurations and analyzing scalability are critical, as the parameters that most significantly affect runtime include the number of layers in the multilayer structure, the length of the wavelength array, and the length of the angle of incidence array. Each of these parameters can substantially increase the computational load. To benchmark the performance of the TMMax library, we used the tmm library developed by Steven Byrnes, which is implemented in Python using the NumPy, as the baseline. In all comparisons, both libraries receive identical inputs; that is, the same layer structure, material parameters, wavelength arrays, and angle arrays, ensuring a direct and fair performance comparison.

To analyze the effect of increasing the number of layers on computational performance, we sample 20 different multilayer structures ranging from 2-layer to 400-layer. As the number of layers grows, the complexity of the multilayer structure correspondingly increases, impacting the computation time required for the simulation. The material of each layer is assigned by randomly picking from seven materials. The thickness of each individual layer was randomly selected within the range of 100 nm to 500 nm. The spectral domain was sampled with a wavelength array consisting of 20 uniformly spaced points spanning from 500 nm to 1000 nm, covering the visible to near-infrared region. Similarly, the angular domain was discretized with 20 evenly distributed incidence angles ranging from normal incidence (0 radians) up to grazing incidence ($\pi/2$ radians). Throughout the study, these spectral and angular parameters were held constant to systematically analyze the impact of increasing the number of layers on the calculation of optical response. This approach enables a clear assessment of how structural

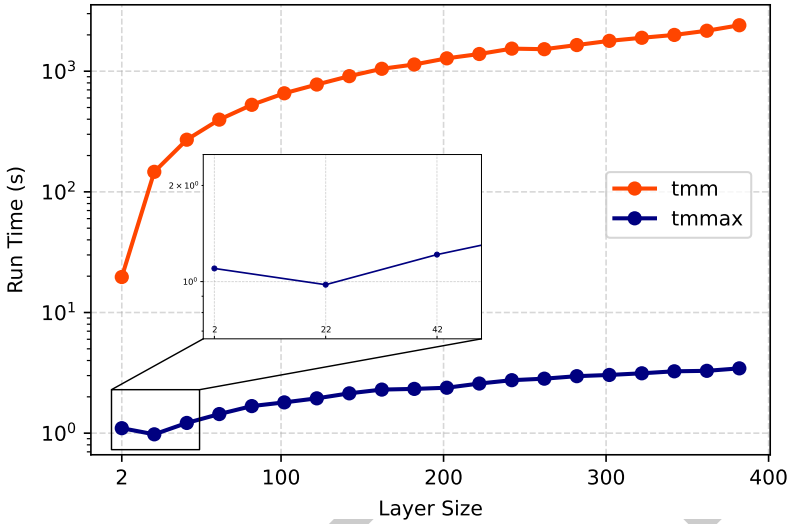


**Figure 3:** Run time as a function of layer size, shown on a semi-logarithmic scale, comparing the tmm package (orange) with TMMax (blue). While the tmm runtime increases steeply with the number of layers, TMMax exhibits significantly improved scalability, with runtime growing slowly and eventually flattening for large stacks. The inset highlights that TMMax maintains a nearly constant execution time (∼1.0–1.2 s) for small layer counts (2, 22, 42). As the number of layers increases, the performance advantage of TMMax becomes more pronounced, reaching a speedup of approximately 18× at 2 layers and up to 700× at 400 layers.

complexity, via layer count, influences computational demand and optical characteristics independently of spectral or angular sampling variations. As shown in Figure 3, the runtime of the tmm increases rapidly as the number of layers increases, whereas TMMax increases much more slowly and exhibits a more scalable behavior. In the inset of Figure 3, the runtime of TMMax remains nearly constant (∼1.0 to ∼1.2 seconds) for low layer numbers (2, 22, 42). Additionally, as the number of layers increases, the speedup provided by TMMax increases. As can be seen in Figure 3, TMMax achieves approximately an 18-fold speedup for a 2-layer structure, while this acceleration reaches up to 700-fold for 400 layers.

Another important factor is the lengths of the wavelength and incident angle arrays. To benchmark the effects of these parameters, we perform tests analogous to the layer size benchmark configuration described above. However, this time, keeping the number of layers constant, we sample the sizes of the wavelength and angle arrays at 20 different values ranging from 2 to 100, and for each combination, we create simulation grids ranging from 2×2 to 100×100. In tests conducted for an 8-layer structure (Figure 4a), the runtime of the tmm increases rapidly as the grid size grows, reaching approximately 138 seconds for the 100×100 configuration. In contrast, TMMax exhibits resilience against this increase, remaining below 3 seconds across the entire grid. However, in the 2×2 grid of the two colormaps, we observe that tmm outperforms TMMax. This difference stems from the underlying implementations: tmm is implemented using NumPy, whereas TMMax is built upon JAX. Generally, NumPy achieves faster execution for small input sizes due to its minimal per-operation dispatch overhead, whereas JAX incurs a higher initialization cost that can adversely affect performance in such scenarios. This explains why the tmm code runs around 0.1 seconds and the TMMax code runs approximately 0.6 seconds. As shown in Figure 4b, the performance advantage of TMMax becomes significantly more evident when the number of layers reaches 80. While tmm's runtime exceeds 760 seconds, TMMax remains below 8 seconds across the entire grid. This demonstrates that TMMax exhibits high efficiency and stability against both problem size and structural complexity.

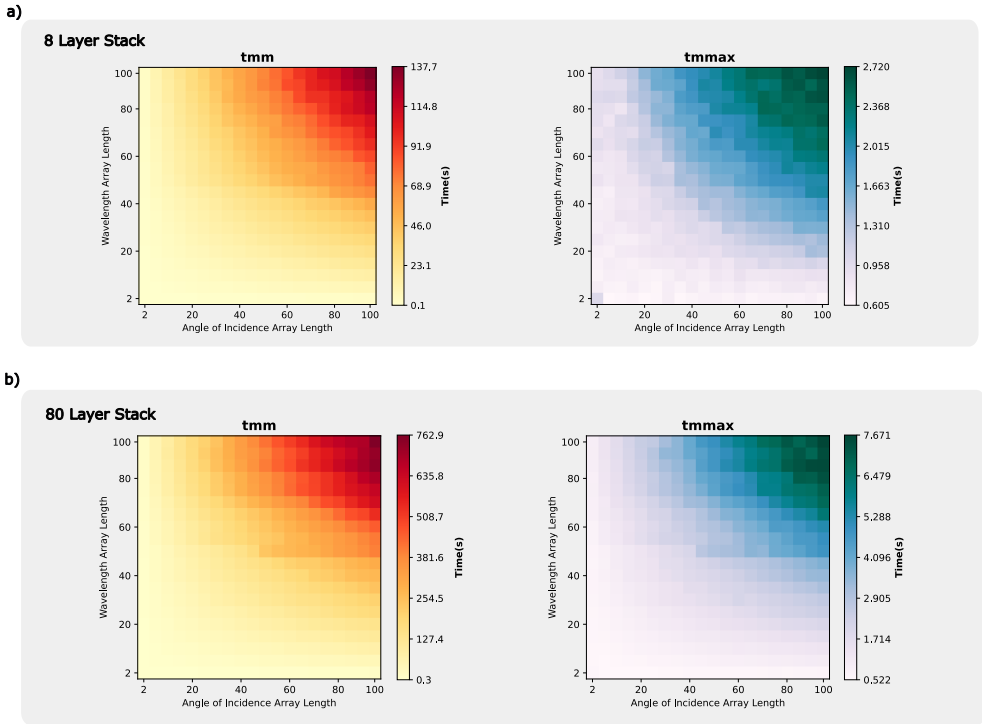


**Figure 4:** The colormaps show the runtime performance of the tmm package and TMMax across varying simulation grid sizes, defined by the lengths of the wavelength and angle arrays. For an 8-layer stack (a), tmm's runtime increases significantly with grid size, while TMMax remains consistently faster; for small inputs, tmm shows a slight advantage over TMMax. A more pronounced performance gap appears in the 80-layer stack (b), where TMMax consistently completes simulations in under 8 seconds, while tmm exceeds 760 while tmm exceeds 760 seconds, highlighting the enhanced scalability and efficiency of TMMax in large-scale thin-film simulations.

To ensure the reliability of the comparisons, we use Python's built-in timeit module. Each simulation repeats 50 times, and the average value is taken; thus, transient systematic effects minimize. The input parameters, including material sequences, layer thicknesses, wavelength values, and angle arrays, are kept the same for both libraries. The comparisons run on a single Intel Core i9 processor core only, without using any GPU or multicore processor, to ensure a fairer comparison.

# Acknowledgements

This work was supported by the Scientific and Technological Research Council of Türkiye (TUBITAK) under the 2209-A Research Project Support Programme for Undergraduate Students, 2022 First-Term Call.

# References

Abelès, Florin. (1950). Recherches sur la propagation des ondes électromagnétiques sinusoïdales dans les milieux stratifiés - application aux couches minces. *Ann. Phys.*, *12*(5), 596–640. https://doi.org/10.1051/anphys/195012050596

Acquaroli, L. (2022). *Lnacquaroli/ThinFilmsTools.jl: v0.9.0* (Version v0.9.0). Zenodo. https://doi.org/10.5281/zenodo.6479354

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, *18*(153), 1–43.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/jax-ml/jax

Byrnes, S. J. (2020). *Multilayer optical calculations*. https://arxiv.org/abs/1603.02720

Centurioni, E. (2005). Generalized matrix method for calculation of internal light energy flux in mixed coherent and incoherent multilayers. *Appl. Opt.*, *44*(35), 7532–7539. https://doi.org/10.1364/AO.44.007532

DeepMind, Babuschkin, I., Baumli, K., Bell, A., Bhupatiraju, S., Bruce, J., Buchlovsky, P., Budden, D., Cai, T., Clark, A., Danihelka, I., Dedieu, A., Fantacci, C., Godwin, J., Jones, C., Hemsley, R., Hennigan, T., Hessel, M., Hou, S., … Viola, F. (2020). *The DeepMind JAX Ecosystem*. http://github.com/google-deepmind

Dmitriev, P. (2017). *PyTMM: Python implementation of the transfer matrix method for optical simulations* (Version v1.0.1c). https://doi.org/10.5281/zenodo.1063716

Dominec, F. (2017). *Transfer_matrix_method: Python implementation of the transfer matrix method*. https://github.com/FilipDominec/transfer_matrix_method.

Dyakov, S. A., Tolmachev, V. A., Astrova, E. V., Tikhodeev, S. G., Timoshenko, V. Yu., & Perova, T. S. (2010). Numerical methods for calculation of optical properties of layered structures. In K. A. Valiev & A. A. Orlikovsky (Eds.), *International conference on micro- and nano-electronics 2009* (Vol. 7521, p. 75210G). International Society for Optics; Photonics; SPIE. https://doi.org/10.1117/12.862566

Harbecke, B. (1986). Coherent and incoherent reflection and transmission of multilayer structures. *Applied Physics B*, *39*(3), 165–170.

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hegde, R. S. (2019). Accelerating optics design optimizations with deep learning. *Optical Engineering*, *58*(6), 065103. https://doi.org/10.1117/1.OE.58.6.065103

Katsidis, C. C., & Siapkas, D. I. (2002). General transfer-matrix method for optical multilayer systems with coherent, partially coherent, and incoherent interference. *Applied Optics*, *41*(19), 3978–3987.

Kingma, D. P., & Ba, J. (2017). *Adam: A method for stochastic optimization*. https://arxiv.org/abs/1412.6980

Kness, M. (2008). ColorPy-a python package for handling physical descriptions of color and light spectra. *The Source Package Can Be Found at Https://Github. Com/Markkness/ColorPy*.

Liu, D. C., & Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, *45*(1), 503–528.

Luce, A., Mahdavi, A., Marquardt, F., & Wankerl, H. (2022). TMM-fast, a transfer matrix computation package for multilayer thin-film optimization: tutorial. *J. Opt. Soc. Am. A*, *39*(6), 1007–1013. https://doi.org/10.1364/JOSAA.450928

Maclaurin, D., Duvenaud, D., & Adams, R. P. (2015). Autograd: Effortless gradients in numpy. *ICML 2015 AutoML Workshop*, *238*.

DRAFT

Macleod, H. A., & Macleod, H. A. (2010). *Thin-film optical filters*. CRC press.

Nishida, K., Ito, Y., & Nakano, K. (2011). Accelerating the dynamic programming for the matrix chain product on the GPU. *2011 Second International Conference on Networking and Computing*, 320–326. https://doi.org/10.1109/ICNC.2011.62

OpenXLA Team. (2023). *XLA: Accelerated Linear Algebra Compiler for Machine Learning*. https://github.com/openxla/xla. https://github.com/openxla/xla

Polyanskiy, M. N. (2024). Refractiveindex. Info database of optical constants. *Scientific Data*, *11*(1), 94.

Santbergen, R., Smets, A. H. M., & Zeman, M. (2013). Optical model for multilayer structures with coherent, partly coherent and incoherent layers. *Opt. Express*, *21*(S2), A262–A267. https://doi.org/10.1364/OE.21.00A262

Shi, Y., Li, W., Raman, A., & Fan, S. (2017). Optimization of multilayer optical films with a memetic algorithm and mixed integer programming. *Acs Photonics*, *5*(3), 684–691.

DRAFT